Beyond First-Order Methods for Distributed Optimization

Chengchang Liu

May 4, 2023

Outline

1. Problem Setup
2. Hardness for Second-Order Distributed Optimization
3. C2EDEN: Communication and Computation Efficient Distributed Newton Method
4. Future Work

Outline

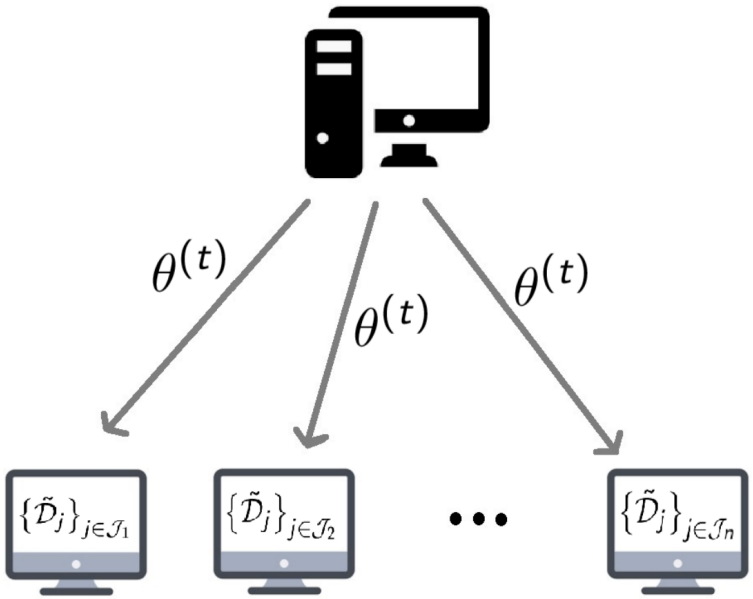
1. Problem Setup
2. Hardness for Second-Order Distributed Optimization
3. C2EDEN: Communication and Computation Efficient Distributed Newton Method
4. Future Work

Settings

We consider the **centralized** distributed second-order optimization.

$$\min_{\mathbf{x}\in\mathbb{R}^d} f(\mathbf{x}) = \frac{1}{n}\sum_{i=1}^{n} f_i(\mathbf{x}), \tag{1}$$

where n is the numbers of the devices.



Single agent optimization

We first consider single-agent optimization of the form

$$\min_{\mathbf{x}\in\mathbb{R}^d} f(\mathbf{x}). \tag{2}$$

We define the oracle

$$T_{p,L_p}(\mathbf{x}) := \arg\min_{\mathbf{y}} \underbrace{f(\mathbf{x}) + \sum_{i=1}^{p} \frac{1}{i!}\nabla^i f(\mathbf{x})[\mathbf{y}-\mathbf{x}]^i}_{f_p(\mathbf{y};\mathbf{x})\text{:Taylor expansion at } \mathbf{x}} + \frac{L_p}{(p+1)!}\|\mathbf{y}-\mathbf{x}\|^{p+1}.$$

Proposition

If $\nabla^p f(\mathbf{x})$ is L_p-Lipschitz continuous, i.e.,

$$\|\nabla^p f(\mathbf{x}) - \nabla^p f(\mathbf{y})\| \le L_p\|\mathbf{x}-\mathbf{y}\|,$$

then we always have $f(\mathbf{y}) \le f_p(\mathbf{y};\mathbf{x})$.

First/Second-order methods for single agent optimization

$$T_{p,L_p}(\mathbf{x}) := \arg\min_{\mathbf{y}} f(\mathbf{x}) + \sum_{i=1}^{p} \frac{1}{i!}\nabla^i f(\mathbf{x})[\mathbf{y}-\mathbf{x}]^i + \frac{L_p}{(p+1)!}\|\mathbf{y}-\mathbf{x}\|^{p+1}.$$

- First-order method:

$$\mathbf{x}_{t+1} = T_{1,L_1}(\mathbf{x}_t) = \mathbf{x}_t - \frac{1}{L_1}\nabla f(\mathbf{x}_t)$$

- Second-order method:

$$\mathbf{x}_{t+1} = T_{2,L_2}(\mathbf{x}_t) = \mathbf{x}_t - \left(\nabla^2 f(\mathbf{x}_t) + L_2 r_t \mathbf{I}_d\right)^{-1} \nabla f(\mathbf{x}_t)$$

for the simplicity, we write Newton method in the following form in the left talk

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \left(\nabla^2 f(\mathbf{x}_t)\right)^{-1} \nabla f(\mathbf{x}_t)$$

First/Second-order methods for single agent optimization

Second-order methods enjoy a superior convergence rates than the first-order methods!

Convergence behavior of first and second-order methods

Methods	Nonconvex	Strongly-convex
Gradient descent	$\mathcal{O}(\epsilon^{-2})$	linear
(Cubic-regularized) Newton	$\mathcal{O}(\epsilon^{-3/2})$	superlinear

Outline

1. Problem Setup
2. Hardness for Second-Order Distributed Optimization
3. C2EDEN: Communication and Computation Efficient Distributed Newton Method
4. Future Work

First-order methods for distributed optimization

Let's return to the centralized distributed optimization.

$$\min_{\mathbf{x}\in\mathbb{R}^d} f(\mathbf{x}) = \frac{1}{n}\sum_{i=1}^{n} f_i(\mathbf{x}) \tag{3}$$

Distributed gradient descent is very popular for many tasks.

- The client i: calculate $\nabla f_i(\mathbf{x}_t)$
- The server: $\mathbf{x}_{t+1} = \mathbf{x}_t - \frac{1}{L_1}\underbrace{\left(\frac{1}{n}\sum_{i=1}^{n}\nabla f_i(\mathbf{x}_t)\right)}_{\text{received from each client}} = \mathbf{x}_t - \frac{1}{L_1}\nabla f(\mathbf{x}_t)$

Communication cost at each iteration: $\mathcal{O}(d)$ $(\nabla f_i(\mathbf{x}_t), \mathbf{x}_{t+1})$

Hardness for second-order for distributed optimization

Consider distributed (cubic-regularized) Newton method.

- The client i: calculate $\nabla f_i(\mathbf{x}_t)$, $\nabla^2 f_i(\mathbf{x}_t)$
- The server:

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \left(\underbrace{\frac{1}{n}\sum_{i=1}^{n} \nabla^2 f_i(\mathbf{x}_t)}_{\text{received from each client}} \right)^{-1} \left(\underbrace{\frac{1}{n}\sum_{i=1}^{n} \nabla f_i(\mathbf{x}_t)}_{\text{received from each client}} \right)$$
$$= \mathbf{x}_t - [\nabla^2 f(\mathbf{x}_t)]^{-1} \nabla f(\mathbf{x}_t)$$

Communication cost at each iteration $\mathcal{O}(d^2)$ ($\nabla^2 f_i(\mathbf{x}_t)$, $\nabla f_i(x_t)$ and $\mathbf{x}_{t+1}$)
Communicating the Hessian is bad for distributed optimization!

Designing communication efficient second-order methods

Communication Efficient: At each iteration, the communication cost between the server and the client is $\mathcal{O}(d)$.

Local Descent (GIANT/Local-Cubic):

- The client i: $\mathbf{x}_{i,t+1} = \mathbf{x}_t - \nabla^2 f_i(\mathbf{x}_t)^{-1} \nabla f(\mathbf{x}_t)$
- The server: $\mathbf{x}_{t+1} = \underbrace{\frac{1}{n}\sum_{i=1}^{n} \mathbf{x}_{i,t+1}}_{\text{received from each client}} \neq \mathbf{x}_t - [\nabla^2 f(\mathbf{x}_t)]^{-1} \nabla f(\mathbf{x}_t)$

These methods do not enjoy the superiorty of second-order methods (neither $\mathcal{O}(\epsilon^{-3/2})$ for non-convex nor superlinear for strongly-convex).

Designing communication efficient second-order methods

Solve quadratic sub-problems: The (Cubic)-Newton step can be reformulated to

$$\min_{\mathbf{y}} f(\mathbf{x}_t) + \langle \nabla f(\mathbf{x}_t), \mathbf{y} - \mathbf{x}_t \rangle + \frac{1}{2}(\mathbf{y} - \mathbf{x}_t)^\top \nabla^2 f(\mathbf{x}_t)(\mathbf{y} - \mathbf{x}_t), \quad (4)$$

The gradient of equation (4) at $\mathbf{y}$ is

$$\nabla f(\mathbf{x}_t) + \nabla^2 f(\mathbf{x}_t)(\mathbf{y} - \mathbf{x}_t) = \frac{1}{n}\sum_{i=1}^{n} \nabla f_i(\mathbf{x}_t) + \left(\frac{1}{n}\sum_{i=1}^{n} \nabla^2 f_i(\mathbf{x}_t)\right)(\mathbf{y} - \mathbf{x}_t)$$

which can be communicated in $\mathcal{O}(d)$.

Solving the quadratic sub-problem at each iteration will increase the total cost and leads to complicated implementation.

Designing communication efficient second-order methods

Compressed Newton methods: Doing compression before sending the second-order information to the server. These methods enjoy local superlinear rate, however

- it requires constructing local Hessian estimator at each client which raise the space complexity to $\mathcal{O}(d^2)$
- No theoretical results for non-convex case.

Designing communication efficient second-order methods

Method	Communication	Space	NC	SC
Distributed-GD	$\mathcal{O}(d)$	$\mathcal{O}(d)$	$\mathcal{O}(\epsilon^{-2})$	linear
Distributed-Newton	$\mathcal{O}(d^2)$	$\mathcal{O}(d^2)$	$\mathcal{O}(\epsilon^{-3/2})$	superlinear
GIANT/DANE	$\mathcal{O}(d)$	$\mathcal{O}(d)$	$\mathcal{O}(\epsilon^{-2})$	linear
Distributed sub Newton	$\mathcal{O}(d)$	$\mathcal{O}(d)$	unknown	linear
Compressed Newton	$\mathcal{O}(d)$	$\mathcal{O}(d^2)$	unknown	superlinear

Outline

1. Problem Setup
2. Hardness for Second-Order Distributed Optimization
3. C2EDEN: Communication and Computation Efficient Distributed Newton Method
4. Future Work

C2EDEN

We developed Communication and Computation Efficient Distributed Newton Method which fixed the previous gaps.

Method	Communication	Space	NC	SC
Distributed-GD	$\mathcal{O}(d)$	$\mathcal{O}(d)$	$\mathcal{O}(\epsilon^{-2})$	linear
Distributed-Newton	$\mathcal{O}(d^2)$	$\mathcal{O}(d^2)$	$\mathcal{O}(\epsilon^{-3/2})$	superlinear
GIANT/DANE	$\mathcal{O}(d)$	$\mathcal{O}(d)$	$\mathcal{O}(\epsilon^{-2})$	linear
Distributed sub Newton	$\mathcal{O}(d)$	$\mathcal{O}(d)$	unknown	linear
Compressed Newton	$\mathcal{O}(d)$	$\mathcal{O}(d^2)$	unknown	superlinear
C2EDEN	$\mathcal{O}(d)$	$\mathcal{O}(d)$	$\mathcal{O}(\epsilon^{-3/2})$	**superlinear**

Observations

- Communicate a full Hessian is difficult, but we can atomize the Hessian

$$\nabla^2 f_i(\tilde{\mathbf{x}}) = \begin{bmatrix} | & | & & | \\ \nabla^2 f_i(\tilde{\mathbf{x}})\mathbf{e}_1 & \nabla^2 f_i(\tilde{\mathbf{x}})\mathbf{e}_2 & \cdots & \nabla^2 f_i(\tilde{\mathbf{x}})\mathbf{e}_d \\ | & | & & | \end{bmatrix}$$

  where communicating $\nabla^2 f_i(\tilde{\mathbf{x}})\mathbf{e}_p \in \mathbb{R}^d$ (a Hessian-vector) requires only $\mathcal{O}(d)$.
- We can communicate a full Hessian in d iterations ($\frac{d^2}{d} = d$)
- We can communicate the full Hessian in d iterations while updating the model!

C2EDEN: algorithm

At iteration $k = td, td+1 \cdots, (t+1)d-1$,

- The client: compute $\nabla f_i(\mathbf{x}_k)$, $\nabla^2 f_i(\mathbf{x}_{td})\mathbf{e}_{k-td+1}$
- The server:

$$\mathbf{H}(:, k-td+1) = \underbrace{\frac{1}{n}\sum_{i=1}^{n} \nabla^2 f_i(\mathbf{x}_{td})\mathbf{e}_{k-td+1}}_{\text{from current iteration}}$$

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \left[\underbrace{\nabla^2 f(\mathbf{x}_{(t-1)d})}_{\text{from iteration } (t-1)d \text{ to } td-1}\right]^{-1} \left(\underbrace{\frac{1}{n}\sum_{i=1}^{n} \nabla f_i(\mathbf{x}_k)}_{\text{from current iteration}}\right)$$

C2EDEN: convergence results

Theorem (Nonconvex case)

Suppose the Hessian of the object function f is L_2-Lipschitz continuous, then with no more than $\mathcal{O}(\sqrt{dL_2}\epsilon^{-3/2})$, C2EDEN finds a stationary point $\mathbf{x}_i$ satisfied

$$\|\nabla f(\mathbf{x}_i)\| \leq \epsilon \quad \textit{and} \quad \lambda_{\min}(\nabla^2 f(\mathbf{x}_i)) \geq -\sqrt{dL_2\epsilon}$$

Theorem (Strongly case)

Suppose the Hessian of the object function f is L_2-Lipschitz continuous and f is μ-strongly convex, then C2EDEN converges at a superlinear rates locally, i.e.,

$$\lim_{k\to\infty} \frac{\|\nabla f(\mathbf{x}_{k+1})\|}{\|\nabla f(\mathbf{x}_k)\|} = 0$$

C2EDEN: empirical study

We validate C2EDEN on regularized logistic regression.

$$f_i(\mathbf{x}) = \frac{1}{m_i} \sum_{j=1}^{m_i} \ln \left(1 + \exp\left(-b_{ij} \mathbf{x}^\top \mathbf{a}_{ij}\right)\right) + \lambda R(\mathbf{x}), \tag{5}$$

For the nonconvex case, we set

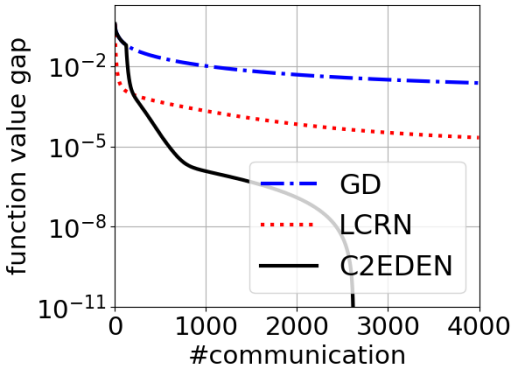
$$R(\mathbf{x}) = \sum_{p=1}^{d} \frac{x_{(p)}^2}{1 + x_{(p)}^2}$$

For the strongly convex case, we set

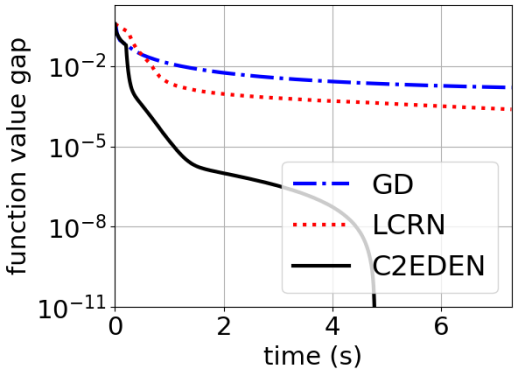
$$R(\mathbf{x}) = \|\mathbf{x}\|_2^2$$

C2EDEN: empirical study

Nonconvex case:



(a) #communication vs. gap

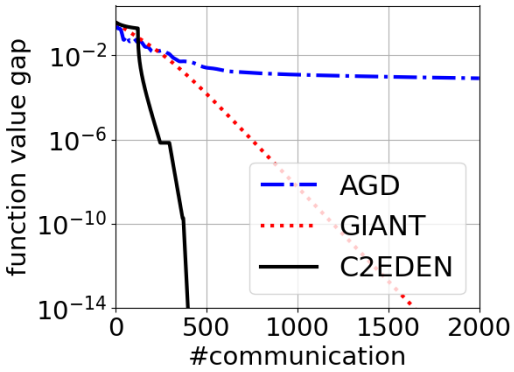


(b) time (s) vs. gap

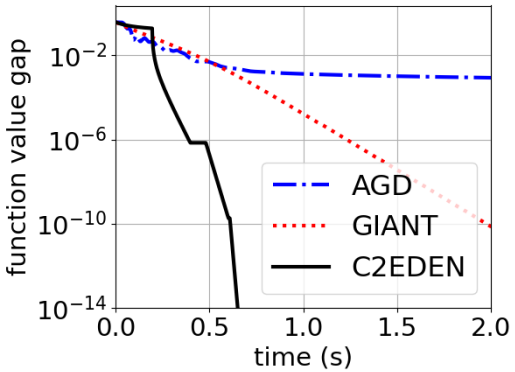
The results of the model of nonconvex regularized logistic regression on "a9a".

C2EDEN: empirical study

Strongly-convex case:



(a) #communication vs. gap



(b) time (s) vs. gap

The results of the model of l_2-regularized logistic regression on "a9a".

Outline

1. Problem Setup
2. Hardness for Second-Order Distributed Optimization
3. C2EDEN: Communication and Computation Efficient Distributed Newton Method
4. Future Work

Future Direction of C2EDEN

C2EDEN takes small but important step for designing beyond first-order distributed optimization methods. We hope this work can motivate the future work such as:

- Can we design communication and computation efficient second-order methods for **decentralized** optimization?
- Can we design communication and computation efficient higher-order methods for **minimax optimization** or **bilevel optimization**?
- Can we take the advantage of the structure of the Neural Network and the framework of C2EDEN and design efficient distributed optimization methods for training large models?

Q & A

Reference

- Communication Efficient Distributed Newton Method with Fast Convergence Rates. In submission.